Chatbot 扩展插件-对接指南

Chatbot 扩展插件可通过独立 iframe 页面集成到第三方业务系统中，为业务系统提供恒脑安全垂域大模型交互能力。

快速上手

Chatbot 扩展插件基于开放接口获取应用凭证，并通过 iframe 嵌入目标页面。

使用步骤：

1. 服务端调用接口获取应用凭证 token。

```
curl --location --request POST 'http://[ip:port]/open/api/assistants/token' \
--header 'Content-Type: application/json' \
--header 'appKey: hengnaoxxx' \
--header 'sign: 169984xxx' \
--data-raw '{
    "userId": "eyJhbGcixxx"
}'
```

2. Web 端使用 token 构造 iframe 地址，并嵌入目标页面：

```
<iframe src="http://[ip:port]/chatbot?appType=assistants&token=xxx" width="100%"
height="100%"></iframe>
```

3. 服务端可通过下文接口注销 token 或校验 token，处理主动登出、有效期刷新等场景。

4. Web 端可结合 iframe 参数和通信事件处理认证过期、报错提示等交互体验。

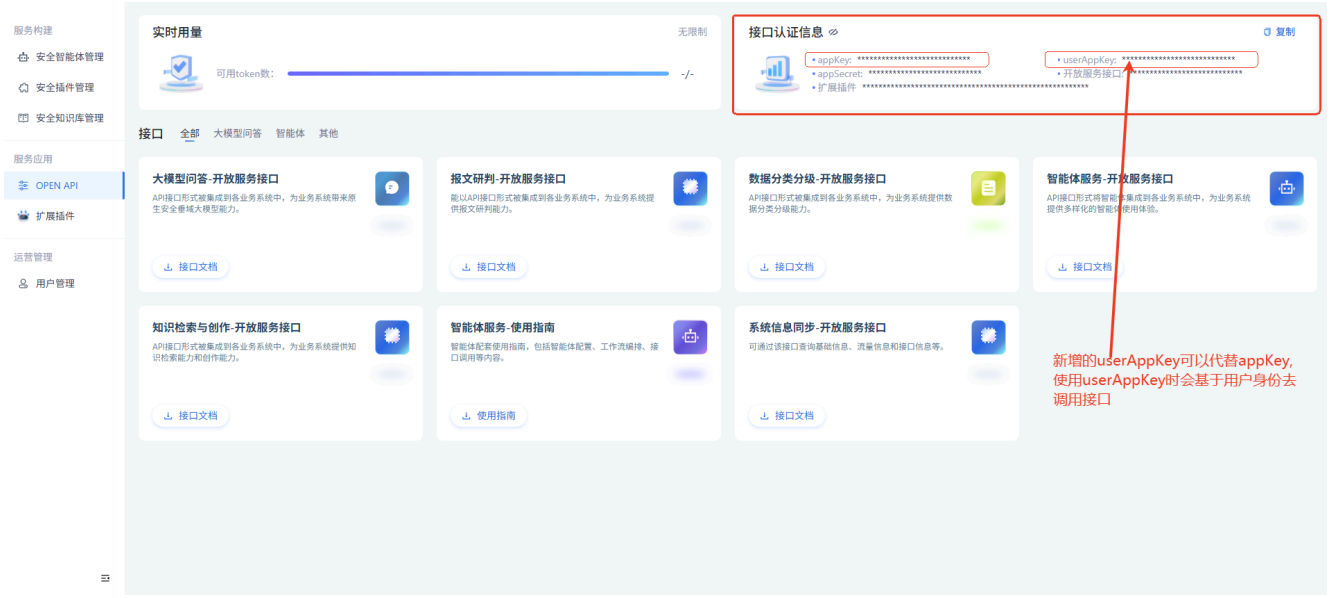
开放接口

接口地址：在“接口认证信息”处获取。

鉴权认证

在使用 API 接口前，您需要从平台获取服务能力的 `appKey` 和 `appSecret`。其中 `appKey` 是应用标识，`appSecret` 是应用密钥，用于生成认证签名 `sign`。

1. 获取 `appKey` 和 `appSecret`：



2. 生成 `sign`：

基于 HmacSHA256 算法，将当前时间戳作为盐值，对 `appKey` 和 `appSecret` 进行哈希计算，生成认证签名 `sign`。

示例（Java）：

```
public static String getSign(String key, String secret) {
    long timestamp = System.currentTimeMillis();
    String data = String.format("%d\n%s\n%s", timestamp, secret, key);
    Mac hmacSHA256 = Mac.getInstance("HmacSHA256");
    SecretKeySpec secretKeySpec = new
SecretKeySpec(secret.getBytes(StandardCharsets.UTF_8), "HmacSHA256");
    hmacSHA256.init(secretKeySpec);
    byte[] sign = hmacSHA256.doFinal(data.getBytes(StandardCharsets.UTF_8));
    return String.format("%d%s", timestamp, new String(new
org.apache.commons.codec.binary.Base64().encode(sign), StandardCharsets.UTF_8));
}
```

示例（Python）：

```
import hmac
import hashlib
import base64
import time

def get_sign(key, secret):
    timestamp = int(time.time() * 1000)
    data = f"{timestamp}\n{secret}\n{key}"
    hmac_sha256 = hmac.new(secret.encode('utf-8'), data.encode('utf-8'), hashlib.sha256)
    sign = hmac_sha256.digest()
    return f"{timestamp}{base64.b64encode(sign).decode('utf-8')}"
```

3. 访问凭证：

   此时，`appKey` 和 `sign` 即为访问凭证。请勿在接口调用时携带 `appSecret`，也不要将 `appKey`、`appSecret` 或 `sign` 硬编码到客户端程序。

接口列表

- 获取 token：`/open/api/assistants/token`
- 注销 token：`/open/api/assistants/token/delToken`
- 校验 token：`/open/api/assistants/token/checkWithRefresh`

公共认证参数

名称	类型	必填	描述
appKey	string	是	应用标识
sign	string	是	签名认证

状态码说明

请求响应的 `code` 含义见下文“状态码表格”。

FLAG 状态码：服务调用反馈标识

后续接口返回的 `flag` 状态码含义：

1. 标识为 `"0"`：表示本次调用成功，无需其他操作。
2. 标识为 `"1"`：表示本次调用请求处理失败，无需其他操作，可以直接跳过。
3. 标识为 `"2"`：表示调用的服务异常，请停止调用，等待一段时间后重试。

示例：其中的 `flag` 等于 `2` 表示当前调用出错，并且不能直接跳过。

```
{
  "msg": "签名验证失败，请查看本地服务器时间是否正确。 当前服务器时间:2024-07-02 10:42:21",
  "code": -402,
  "flag": 2,
  "data": null
}
```

1. 获取 token

调用本接口获取 Chatbot iframe 访问凭证 token。

基本信息

请求地址：`/open/api/assistants/token`

请求方法：POST

Body 参数

名称	类型	必填	描述
userId	string	是	集成方用户唯一身份标识

响应说明

名称	类型	描述
code	string	状态码，`0` 成功，非 `0` 失败
msg	string	状态码描述
data	string	应用凭证 token

示例

请求示例（curl）：

```
curl --location --request POST 'http://[ip:port]/open/api/assistants/token' \
--header 'Content-Type: application/json' \
--header 'appKey: hengnaoxxx' \
--header 'sign: 169984xxx' \
--data-raw '{
    "userId": "eyJhbGcixxx"
}'
```

返回示例：

```
{
  "code": 0,
  "msg": "成功",
  "data": "eyJhbGcixxx"
}
```

2. 注销 token

调用本接口注销应用凭证 token。

基本信息

请求地址：`/open/api/assistants/token/delToken`

请求方法：POST

Body 参数

名称	类型	必填	描述
token	string	是	应用凭证 token

响应说明

名称	类型	描述
code	string	状态码，`0` 成功，非 `0` 失败
msg	string	状态码描述
data	string	空

示例

请求示例（curl）：

```
curl --location --request POST 'http://[ip:port]/open/api/assistants/token/delToken' \
--header 'Content-Type: application/json' \
--header 'appKey: hengnaoxxx' \
--header 'sign: 169984xxx' \
--data-raw '{
    "token": "eyJhbGcixxx"
}'
```

返回示例：

```
{
  "code": 0,
  "msg": "成功",
  "data": null
}
```

3. 校验 token

调用本接口校验应用凭证 token 的合法性，可按需刷新 token 有效期。

基本信息

请求地址：`/open/api/assistants/token/checkWithRefresh`

请求方法：POST

Body 参数

名称	类型	必填	描述
token	string	是	应用凭证 token
refresh	boolean	否	是否刷新 token 有效期，默认为 `true`

响应说明

名称	类型	描述
code	string	状态码，`0` 成功，非 `0` 失败
msg	string	状态码描述
data	boolean	token 是否有效，`true` 有效，反之无效

示例

请求示例（curl）：

```
curl --location --request POST 'http://[ip:port]/open/api/assistants/token/checkWithRefresh' \
--header 'Content-Type: application/json' \
--header 'appKey: hengnaoxxx' \
--header 'sign: 169984xxx' \
--data-raw '{
    "token": "eyJhbGcixxx",
    "refresh": true
}'
```

返回示例：

```
{
  "code": 0,
```

```
  "msg": "成功",
  "data": true
}
```